Projektuppgift

DT211G Frontend-baserad webbutveckling

Daniela Giovanella

Mittuniversitetet

MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. **Campus Sundsvall** Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00. Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Daniela Giovanella, dagi2500@student.miun.se
Utbildningsprogram: Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: VT, 2026

Sammanfattning

I detta projekt har en webbsida utvecklats med syfte att presentera information om spelserien Far Cry. Detta görs genom att kombinera data från flera olika källor genom API: er. Webbsidan är uppbyggd som en mashup där speldata hämtas från RAWG API som sedan även kompletteras med sammanfattningar av spelen med information tagen från Wikipedia API.

Webbsidan presenterar de högst rankade spelen, men gör det även möjligt för användare att själva söka bland de olika titlarna. Innehållet uppdateras och visas dynamiskt med JavaScript.

I projektet används en automatiserad arbetsprocess med Node.js och Vite föra att hantera och effektivisera utveckling, optimering och publicering.

Sass har använts för styling av webbplats där även animationer används för att förbättra användarupplevelsen.

Resultatet är en responsiv webbsida som uppfyller projektets alla mål och visar på hur flera API: er kan kombineras i en modern webbutvecklingsmiljö för att skapa nya tjänster och produkter.

Innehållsförteckning

Sammanfattning iii
1 Introduktion / Inledning 1
1.1 Problemmotivering 1
1.2 Syfte 1
1.3 Mål med projektet 1
2 Teori / Bakgrundsmaterial 2
2.1 Termer 2
3 Metod 5
3.1 Teknik och program 5
3.2 Val av API: er 5
3.3 Datainsamling 5
3.4 Mashup av API: er 5
3.5 Automatiserad arbetsprocess 6
3.6 Versionshantering och publicering 6
3.7 Styling och struktur 6
3.8 Interaktivitet med JavaScript 6
3.9 Responsiv design 6
4 Konstruktion 7
4.1 Målgrupp och grafisk profil 7
4.2 Framtagning av grafiskt material 7
4.3 Grafiska skisser 7
4.4 Skapa projekt med vite 8
4.5 HTML 9
4.6 Sass/scss: Styling 9
4.7 JavaScript: Funktioner 10
4.8 Animationer 11
4.9 Dokumentation 12
5 Resultat 13
6 Analys 14
7 Källförteckning 15
Bilaga A: Grafisk profil 17
Bilaga B: Grafiska skisser 18

Projektuppgift
Daniela Giovanella

2026-03-22

1 Introduktion / Inledning

1.1 Problemmotivering

Inom webbutveckling idag använder man ofta data från flera olika källor för att skapa dynamiska och interaktiva webbplatser och applikationer. I detta projekt används detta arbetssätt för att utveckla en webbsida riktad mot personer med intresse för spel, och specifikt Far Cry-serien, där information hämtas från flera externa källor för att skapa en dynamisk och interaktiv webbsida. Denna typ av lösning, kallad en mashup, kräver att man både kan hantera den tekniska implementationen och arbeta med data från flera olika källor.

1.2 Syfte

Syftet med projektet är att skapa en webbplats för sökning av information om spel ifrån Far Cry-serien genom att kombinera data ifrån olika API: er och på detta sätt kunna erbjuda användaren en tjänst där de kan få en bra överblick över vilka spel som finns samt att kunna ta del över en mer detaljerad information om varje spel i serien.

1.3 Mål med projektet

Projektkrav:

- Skapa en webbplats/applikation som hämtar data från minst två JSON-baserade API: er.
- Skapa en mashup genom att kombinera data från dessa API: er.
- Använda en automatiserad arbetsprocess baserad på Node.js och optimera filer med relevanta verktyg.
- Använda en automatiserad arbetsprocess baserad på Node.js och optimera filer med relevanta verktyg.
- Skapa en responsiv design och en tydlig grafisk profil
- Använda Sass för att strukturera och effektivisera arbetet med styling.
- Implementera animationer och interaktivitet i på webbsidan.

Mål för webbsida:

- Implementera en sökfunktion och dynamisk visning av innehåll med JavaScript.
- Med hjälp av API: er hämta och ta fram data om specifika spel baserat på användarens sökningar.
- Presentera innehållet på ett mot målgruppen tilltalande sätt, anpassat efter spelrelaterat innehåll.

2 Teori / Bakgrundsmaterial

I detta avsnitt presenteras centrala begrepp och tekniker som är relevanta för projektet. Syftet är att ge en grundläggande förståelse för de verktyg, metoder och tekniker som används under utvecklingen av webbsidan.

2.1 Termer

Arbetsprocess

Node.Js – En öppen källkodsplattform och körmiljö som gör det möjligt att köra JavaScript-kod utanför webbläsaren, exempelvis på en server. [1]

Npm och node_modules – npm (Node Package Manager) är ett verktyg för Node.Js som används för att installera och hantera tredjepartsbibliotek, paket och verktys som används i ett projekt. Node_modules är en mapp i ett Node.Js projekt där alla paket och dess beroenden lagras. [2]

Minifiera – Komprimering av textfiler och kod där alla tecken som inte behövs för att webbläsaren ska förstå koden tas bort. [3]

Automatisering - Användning av verktyg för att underlätta utvecklingsprocessen genom att låta dessa verktyg utföra uppgifter som bland annat optimering och minifiering vilket gör processen snabbare. [3]

Vite - Ett byggverktyg som används inom webbutveckling som gör det snabbarbare att utveckla och optimera frontend-applikationer. [4]

Källkodsfil/Publiceringsfil – Källkodsfiler är de filer som arbetas med under utvecklingen av projekt. Här finns originalkoden som strukturerats upp för att vara enkel att förstå och underhålla. Publiceringsfiler är de filer som är redo att publiceras och är på olika sätt optimerade för att ge en bra användarupplevelse så som snabb laddningstid. [3]

JavaScript/Funktionalitet

Sort/filter – Metoder i JavaScript som används för att hantera Arrayer. *sort* används för att sortera element i en viss ordning medan man med *filter* skapar en ny Array utifrån specifika villkor. [5]

Promise – Ett sätt i JavaScript att hantera data som sker asynkront, exempelvis när data hämtas från en server. Den representerar ett resultat som inte är klart. [6]

Async/await – Sätt i JavaScript att skriva asynkron kod på där *async* sätts på en async funktion som returnerar ett Promise, och *await* används för att vänta på resultatet från ett Promise innan koden fortsätter köras. [7]

Try/catch - Sätt i JavaScript att hantera fel bland annat i asynkrona funktioner, där *try* sätts runt den kod som kan ge eventuella fel, och *catch* fångar upp dessa fel. [8]

Styling av webbsida

Preprocessor - Ett verktyg som kan användas för att skriva CSS-kod på ett mer effektiv och strukturerat sätt. Sass är ett sådant verktyg och som kommer användas i detta projekt. [9]

Partials – Delar av Sass-kod som delas upp i separata Sass-filer för bättre struktur för de olika delarna i ett projekt. [9]

Inherit/extend- Ett sätt att låta en CSS-klass ärva delar av från en annan klass för att minska duplicering av kod.[9]

Villkorsoperator (if sats) – Ett verktyg där man kan utföra olika stilar på ett element beroende på villkor. I projektet används if-sats för att använda kod beroende på om villkoret är sant eller ej. [10]

Mixins – Ett block av kod som kan återanvändas på flera selektorer utan att behöva kopiera koden.[9]

Variabler – Namngivna värden på exempelvis färg eller storlek som kan återanvändas i hela Sass-projektet utan att behöva återupprepa koden. [9]

Nesting – Möjlighet att i Sass kunna skriva CSS-regler inuti varandra som kan spegla HTML-strukturen och göra koden mer läsbar.[9]

7-1 mönster – En struktur för Sass-projekt som delar koden (partials) i sju mappar med delade filer med liknande samhörighet. Detta för att göra koden mer organiserad och lättare att underhålla.[11]

Animering

Key frames - Ett sätt i CSS att definiera stegvisa förändringar i en animation, där man anger vilka stilar som ska gälla vid olika punkter under animationen. [12]

Transition – Ett sätt att I CSS skapa mjuka övergångar med olika stilar som exempelvis färg eller bredd, ofta vid hovring över ett element. [13]

Transform – Ett CSS-attribut som används för att förändra ett element. Detta kan bland annat vara att få elementet att rotera, skala upp eller ner i storlek, eller att flytta det, och kombineras ofta med andra animationer. [14]

API/Data

API - Ett gränssnitt som används för kommunikation mellan olika program och system för att exempelvis hämta eller skicka data. [15]

API nyckel – En unik kod som används för att ge åtkomst till ett API vid förfrågningar från en applikation eller projekt. [16]

JSON – Ett dataformat som används för att lagra och överföra data från exempelvis en server till en webbsida. [17]

JSDoc (dokumentation) – Beskrivning av hur kod, funktioner och olika system i JavaScript, vilket hjälper utvecklare att bättre förstå koden. [18]

Mashup - En webbsida eller webbapplikation som använder sig av flera källor för att skapa en tjänst som presenteras i kombination med varandra. [19]

3 Metod

3.1 Teknik och program

I projektet används flera olika tekniker och verktyg för att utveckla webbsidan. *Vite* [4] har används som utvecklingsmiljö tillsammans med *Node.js* [1] för att kunna genomföra arbetet med en automatiserad arbetsprocess. För versionshantering används Git och Github, och som publiceringsplattform används Render använts till både webbsida och dokumentation. För design av webbsidan och framtagning av grafiska skisser har Figma använts, och för grafiskt innehåll vilket består av en bakgrundsbild har photoshop använts för redigering. Google fonts används för att integrera både typsnitt och ikoner.

3.2 Val av API: er

API: er används för kommunikation mellan olika system och för att hämta data, [15] och för att hämta data om Far Cry -spelen så används två olika API: er. RAWG API används för att hämta information så som titel, bilder, rating och releasedatum. [21] Wikipedia API används för att hämta sammanfattningar om spelen. [22] Dessa API: er valdes eftersom dom tillsammans med de data som dom tillför kan ge en komplett övergripande presentation av spelen.

3.3 Datainsamling

Datainsamling från de båda API:.er sker genom AJAX-anrop som är ett sätt att uppdatera sidan utan att hela sidan behöver laddas om. [20] Detta sker i JavaScript där data hämtas i JSON-format. JSON används då det är ett format som är strukturerat på ett sätt som gör det enkelt att läsa av för att överföra data mellan systemen. [17]

För att hantera anropen används *async* och *await* [7] med *Promise* [6], vilket gör det mjöligt att jobba med kod på ett strukturerat sätt. För att hantera eventuella fel vid hämtning av data används *try* och *catch* för att undvika att webbsidan kraschar vid fel. [8]

3.4 Mashup av API: er

Då projektet går ut på att skapa en mashup så används de två API: er för att komplettera varandra. [19] Som grund används RAWG för att hämta relevanta titlar och metadata som används för bland annat sortering av topptitlar. Anropen från dessa API: er baseras på *Promises,* vilket hanteras med *async/await*. API: er returnerar data i JSON format som sedan behandlas i JavaScript. För att skapa en mashup så används Wikipedia API för att hämta summerad information om varje spel utifrån titel.

3.5 Automatiserad arbetsprocess

Genom projektet använd en automatiserad arbetsprocess för att effektivisera utvecklingen. [3] Men hjälp av *Vite* [4] och *Node.js* [1] så sker en automatisk uppdatering av webbsidan under utvecklingen, samt optimering av filerna vid publicering. I projektet arbetas det med källkodsfiler som sedan genom Vite omvandlas till optimerade publiceringsfiler vid build-processen. Denna process inkluderar optimering av filer och andra resurser samt minifiering av CSS och JavaScript-filer för att förbättra prestanda och laddningstid. [3]

3.6 Versionshantering och publicering

Versionshantering av projektet sket med Git och lagras Github vilket gör det möjligt att spåra ändringar, justera fel och tillåter även automatisk uppdatering av publicerad sida via Render vid ändringar i projektet.

3.7 Styling och struktur

För styling av webbsida används *Sass* vilket gör det möjligt att strukturera upp koden och göra denna återanvändbar på ett effektivt sätt genom att dela upp olika delar av webbsidans styling i olika *partials*. [9] Till grund för filstruktur ligger 7-1 mönstret. Även fast filerna inte är uppdelade helt enligt detta mönster så är det bra mönster att följa då detta gör koden mer lättläst och enklare att underhålla. [11]

3.8 Interaktivitet med JavaScript

JavaScript används för att skapa en dynamisk sida för uppdatering av innehåll på webbsidan. För att skapa webbsidan enligt design och göra den interaktiv så används funktioner som sortering och filtrering av data tillsammans med en sökfunktion för att presentera innehåll baserat på design och användarens sökningar.

3.9 Responsiv design

Webbsidan är utvecklad för att fungera responsivt på olika skärmstorlekar.
Detta uppnås genom användning av flexibla mått, anpassad styling och design samt media queries.

4 Konstruktion

4.1 Målgrupp och grafisk profil

Målgrupp för webbsidan är personer med intresse för gaming och framförallt Far Cry spelen eller med en nyfiken på dessa och vill få en bra överblick över de olika titlarna i spelserien. Syftet är att ge dessa användare relevant information om varje titel på ett samlat och lättillgängligt sätt.

Vid framtagning av den grafiska profilen har inspiration hämtats från spelseriens visuella stil som ofta innefattar naturmiljöer som djungel, berg och andra naturrika landskap. Detta tillsammans med en spelstil som är inriktad på att strida. Utifrån detta har en färgpalett skapats med en mörk färg som grund kombinerat med naturnära färger i grönt och orange/rött. Färgvalet syftar till hatt förstärka estetiken från spelen och implementera det på ett visuellt lämpligt sätt på webbsidan. Även för typsnitt har inspiration tagits från spelseriens logotyp för att direkt skapa en igenkänning för användaren. I kontrast med webbsidans mörka grund så används en ljus färg för rubriker och text. Denna färg grundar sig i den gröna färg som används för andra element på sidan. För specifik information om färgkoder och typsnitt se bilaga A, Grafisk profil.

4.2 Framtagning av grafiskt material

Som utgångspunkt för designen har en bakgrundsbild för webbsidan använts. Denna bakgrundsbild är en skärmdump ifrån spelet Far Cry 5 och är tagen av mig själv. Bilden är redigerad för att framhäva de röda och gröna tonerna samt att skapa ett mörkare djup i bilden, detta för att få bilden att passa ihop bra med den grafiska profilen. Tonerna har jämnats ut och bilden har fått en mörk overlay för att göra text och övrigt innehåll väl synligt och lättläst.

Bilden har optimerats genom konvertering till formatet webp och skapats i tre olika storlekar: 1900px bredd för större desktopskärm med en bildstorlek på 634 kb. En storlek för mindre skärmar: 1024px med 176 kb i bildstorlek. En storlek för mobilskärm på 600px i bredd och 52,6 kb i bildstorlek.

Övriga grafiska element på webbsidan är ikoner för sökning och tillbakamarkering som är tagna från Google Fonts Icons och har anpassats efter färg från grafisk profil för textfärg, detta för att skapa enhetlighet. Dessa ikoner har formatet png med båda med en bildstorlek under 1 kb.

En favicon har skapats i photoshop utifrån grafisk profil och genererats i flera storlekar med hjälp av programmet RealfaviconGenerator. Format för favicon i png-format är 96x96px och 180x180px. För möjlighet att kunna använda favicon på flera sätt finns även en större bildstorlek på 512X512px sparad.

4.3 Grafiska skisser

De grafiska skisserna har tagits fram med utgångspunkt i den grafiska profilen, bakgrundsbilden samt hur information från valda API: er är tänkt att presenteras.

Syftet med skisserna är planering av struktur och layout innan kodning av webbsidan påbörjas.

Två varianter av designen togs fram, en standardversion och en version som specifikt är anpassad för de med en ljus skärminställning, med läsbarhet av informationstext i åtanke. Se exempel och utdrag från de grafiska skisserna i bilaga B – Grafiska skisser.

Webbsidan skissades upp i tre lägen:

Startsida – Innehåller en header med logotyp placerad till vänster. Huvudinnehållet på sidan innehåller en sökfunktion och under ett urval av de tre högst rankade spelen presenterade med bild och ranking i form av cards.

Söksida – Visar resultatet av sökningen i ett rutmönster layout där varje spel visas i ett liknande card som i toppspelen.

Informationssida – Visar detaljerad information om ett specifikt spel med bild, fakta och en sammanfattning av spelet hämtad från API: er.

Skisserna har även anpassats för flera skärmstorlekar så som tablet och mobile. Den främsta skillnaden mellan de olika layouterna är hur cardsen presenteras. Vid mindre skärm visas dessa färre i rad, och för mobile visas toppspelen i en kolumn i stället för att säkerställa god läsbarhet och användarvänlighet.

4.4 Skapa projekt med vite

Innan kodning av webbsidan så startades projektet upp med hjälp av *Vite* genom kommandot npm create vite@latest, där alternativet Vanilla valdes.

Mappstrukturen har organiserats för att skapa en tydlig separation mellan de olika delarna i projektet. I *scr*-mapp finns separata mappar för JavaScript och SCSS. En public-mapp används för bilder och ikoner. Där gör de tillgängliga direkt vid publicering.

Då ett av API: er kräver nyckel så har en *.env*-fil skapats för hantering av just känslig information. Denna fil är på grund av dess innehåll exkluderad från versionshantering och ignoreras via *.gitignore*. Det gör även mappen *node_modules* som innehåller alla installerade paket och beroenden. Detta för att inte bidra med onödigt stora filer i repositoryt.

En *package.json*-fil används för att definiera projektets beroenden och används tillsammans med npm för att installera och hantera olika paket.

En konfigurationsfil *vite.config.js* används för at anpassa projektets byggprocess. Då grundtanken var att lägga till ytterligare en HTML-sida i projektet så har en *defineConfig* skapats för att Vite ska kunna läsa av flera sidor. Här har även ett plug in för optimering av bilder i olika format implementerats, *vite-image-optimizer.* Detta för att förbättra prestandan vid laddning av sidan. Webbsidan innehåller inga högupplösta fotobilder vilket gjorde att nivån på komprimering kunde justeras mycket utan att märkbart påverka bilden och nivån är lagt på 70 för både

webp och png format. Detta resulterade i en total minskning av bildstorleken med cirka 93 %, motsvarande en besparing på 878,56 kb av totalt 944,81 kb.

4.5 HTML

Html-strukturen i projektet är uppbyggt som en enkel sidan där innehållet uppdateras dynamiskt med hjälp av JavaScript. Tanken bakom detta är att skapa en applikationsliknande känsla där användaren kan interagera med sidan utan att den behöver ladda om.

Sidan är uppdelad med de semantiska sektionerna: *header, main , section* och *footer.* I headern ligger sidan logotyp och en tagline för att skapa en tydlig identitet för sidan. Logotyp fungerar även som en navigationspunk om användaren vill ladda om sidan.

Huvudinnehållet är uppletat i olika sektioner som motsvarar de olika funktionerna. En sektion innehåller sökfunktion där användaren kan skriva in en sökning på ett spel. Sökresultatet är i sin tur kopplad till en underliggande sektion där detta skrivs ut när denna data hämtats från API.

På samma sätt finns det sektioner som fungerar likadant för att visa toppspelen och sida för detaljerad information och summering av spel.

I footer hittas information om använd data på webbplatsen då RAWG har som krav att de måste refereras till vid användning av deras data.[21]

4.6 Sass/scss: Styling

För styling av webbsidan så har Sass/SCSS använts vilket möjliggör en mer strukturerad och modulär hantering av CSS. Projektets stilfiler är strukturerade och organiserade efter 7-1 mönster där koden delats upp i olika mappar och partials beroende på vilken funktion och samhörighet de har. Då omfattningen på projektet är relativt litet så har dock inte alla delar av detta mönster använts.

Strukturen består av mappar med uppdelare efter ansvarsområden. I abstracts definieras variabler och animationer. Mappen *base* innehåller grundläggande typografi för samtliga textelement. I *components* finns separata filer som behandlar komponenter som bakgrundsbild, knappar, spelkort (cards) och sökfunktionen. Layoutrelaterad kod är även den uppdelad i partials för grundläggande sidlayout, media queries för mindre skärm och light-mode. Samtliga partials kopplas samman i en main partial.

Variabler används för att definiera färger enligt grafisk profil. Detta gör det enkelt att återanvända och justera dessa värden globalt. *Mixins* används i projektet för styling av knappar. Detta för att minska duplicerad kod på dessa element. *Nesting* används främst i större layoutelement som header och footer som gör koden för dessa element mer lättläst. För spelkorten används *@extend* för att ärva grundläggande styling för korten och förenkla och minska på kod. Här används även villkorssatsen *if/else* för att avgöra om spelkorten ska ha en färgad border eller inte. Skillnaden kan ses om man jämför desktop-läget med mobil-läget där spelkorten i mobils läge fått en färgad border. [9]

4.7 JavaScript: Funktioner

JavaScript används i projektet för att hantera datainsamling, dynamiskt innehåll och interaktivitet. Webbsidan är uppbyggd med flera funktioner som tillsammans hämtar, bearbetas och presenterar data från API: er.

För att hämta data om spelen används *fetch* i kombination med *async* och *await* som gör det möjligt att jobba med asynkrona anrop på ett strukturerat sätt. Vid dessa anrop används funktionen *encodeURIComponent* för att säkerställa att den söksträng som användaren skriver in i sökfält formateras korrekt i URL vilket minskar risken för fel vid hämtning av data.

Funktion: getGames – Hämtar data om spel

Denna funktion anropar RAWG API baserat på användarens sökning, och filtrerar resultaten till att bara inkludera relevanta Far Cry-spel. Filtreringen sker genom att exkludera exempelvis specialutgåvor och samlingar för att förbättra relevansen i sökresultatet, se kodexempel - Figur 1.

Sökfunktionen är kopplad till en sökknapp som vid klick kontrollerar att inmatad sökning innehåller sökord innan anrop mot API görs. Samtidigt så uppdateras gränssnittet genom att rensa tidigare innehåll och nya rubriker och innehåll visas därefter. Skulle inmatad information vara felaktig får även användaren ett felmeddelande om detta.

```
        // Filtrerar resultaten så att bara relevanta far cry spel kommer upp vid
sökning const filteredGames = rawgData.results.filter(game => {
            const name = game.name.toLowerCase();
            return name.startsWith("far cry") &&
                !name.includes("edition") &&
                !name.includes("collection") &&
                !name.includes("bundle");
        });
```

Figur 1-Filtering

Funktion: showGames – Visar sökresultat

Resultatet från sökningen visas med funktionen *showGames* där speldata skrivs ut i DOM som klickbara spelkort. Varje kort skapas dynamiskt och innehåller bild, titel och releasedatum. Vid klick på dessa kort anropas funktion för att visa mer detaljerad information om varje spel.

Funktion: getInfo – Mashup av två API

Funktionen *getInfo* triggas när användaren har klickat på ett spelkort och syftar till att visa en mer detaljerad information om spelen. Därför används här både

RAWG API och Wikipedia API för att skapa en vy där data från båda dessa presenteras. Detta görs med ett asynkront anrop baserat på spelets titel, då titelnamnet fungerar att anropa från dessa båda API och få ut olika information. Detta resulterar i en vy som visar data från RAWG med bild, och meta data, och en sammanfattning av spelet från Wikipedia. För att förbättra användarupplevelsen och göra det enklare att navigera finns även en tillbaka-knapp som tar användaren tillbaka till tidigare vy, antingen från startsida eller sökningsida beroende på vart denne kommit ifrån. Detta hanteras genom att olika sektioner döljs eller visas beroende på tidigare interaktioner.

Funktion: getTopGames – Hämta info om top tre spel

Vid sidladdning och det som fungerar som en startsida visar de högst rankade Far Cry-spelen. Detta görs möjligt med funktionen *getTopgames* som hämtar information om spelet från RAWG API och filtrerar fram de högst rankade spelen genom sorteringsmetoden *sort* för att sortera spelen baserat på deras ranking i fallande ordning. De tre högst rankade spelen plockas därefter ut med hjälp av *slice,* se kodexempel Figur 2.

```
const filteredGames = rawgData.results.filter(game =>
    game.name.toLowerCase().includes("far cry") &&
    //exkluderar:
    !game.name.toLowerCase().includes("edition") &&
    !game.name.toLowerCase().includes("collection") &&
    !game.name.toLowerCase().includes("bundle") &&
    !game.name.toLowerCase().includes("dragon")
);
//Sorterar spel så att högst rankade kommer fört
const sortedGames = filteredGames.sort((a, b) => b.rating - a.rating);

// Plockar ut dom tre bäst rankade
const topGames = sortedGames.slice(0, 3);
```

Figur 2 – Sortering

Funktion: showTopGames – Visa de högst rankade spelen

Funktionen *showTopGames* hämtar informationen från *getTopGmaes* och skriver ut denna information som spelkort i DOM. Varje kort innehåller bild, titel och betyg, och även dessa spelkort fungerar som en ingång till funktionen *showInfo* vid klick.

Sammanfattningsvis så används JavaScript på webbsidan för att göra denna dynamisk och interaktiv som uppdateras i realtid baserat på användarens handlingar och med hämtade data från externa tjänster.

4.8 Animationer

För att göra webbsidan mer levande och ge visuella tecken och feedback för funktion och interaktion har animering använts på flera element.

Huvudrubrik/Logotyp

För huvudrubriken används en animering för att simulera att texten skrivs ut steg för steg när sidan laddas. Syftet med denna animation är enbart visuell och för att ge ett levande förstaintryck av sidan.

Knapp-aniamtion

Även knappen i sökfunktionen använder *keyframes* för att skapa en animation. Animationen triggas vid klick med hjälp av JavaScript och använder transformering med *scale* för att förstora knappen och tillfälligt ändra färg. Detta för att ge användaren feedback på att klicket har registrerats, se kodexempel – Figur 3.

```
  @keyframes buttonchange {
 0%   { transform: scale(1); }
 50%  { transform: scale(1.1); background-color: variables.$color-accent; }
 70%  { transform: scale(0.95); background-color: variables.$color-accent;}
 100% { transform: scale(1); }
}
```

Figur 3- Knappanimation

Hover-animationer

Flera hover-animationer används för att göra det tydligare för användaren vilka element som är klickbara. Spelkorten använder *transform: scale* vilket förstorar korten för att visa på att dessa går att klicka. Tillbakaknapp ändrar på ett mjukt sätt färg vid hover med hjälp av *transition* o för att visa att denna är markerad.

Sammanfattningsvis används animationerna på sidan på ett subtilt sätt för att inte distrahera användaren från innehållet men ändå ge tydlig visuell feedback.

4.9 Dokumentation

För att underlätta förståelse och eventuellt underhåll av JavaScript-koden har denna dokumenterats med JSDoc, som är ett standardiserat sätt att dokumentera bland annat funktioner och för att tydligt visa vad dessa gör och vilken data som används. Detta gör det enklare att hålla koll på hur information hanteras från exempelvis API: er. [18] Kommentarer beskriver varje funktion, vad den gör och vilken data den tar emot och vilket resultat den ger.

Alla funktioner som hämtar data från API: er har kommentarer som förklarar vad funktionen gör, exempelvis om den hämtar data eller visar toppspel. Vilken information den tar emot, exempelvis om det är en söksträng eller spelobjekt, och hur den påverkar webbsidan. För listor och objekt beskriver kommentarer vilka objekt som skickas till funktioner, och hur informationen används för att generera innehåll i DOM.

5 Resultat

Webbsida har utvecklats enligt projektets syfte och uppfyller kraven och de egna uppsatta målen. Resultatet är en fungerande webbsida som kombinerar data från två olika API: er och presenterar information om spelserien Far Cry. Sidans innehåll är stilsatt med Sass och animationer används med ett klart syfte.

Funktionalitet
Webbsidan gör det möjligt för användare att söka efter Far Cry- spel via en sökfunktion. Vid sökning hämtas data från RAWG API där resultaten filtreras för att bara visa relevanta spel. Vid val av spel hämtas ytterligare data med sammanfattning om spelen från Wikipedia API, vilket gör att målet att skapa en mashup uppfylls. Vid laddning av sidan visas även de tre högst rankade spelen baserad på data från RAWG.

Presentation av innehåll
Informationen presenteras i form av cards som innehåller bild, titel och beroende på om det är sökresultat eller toppspel även ranking eller releasedatum. Vid klick på ett card så visas en mer detaljerad vy med en sammanfattning av spelet. Gränssnittet uppdateras dynamiskt med JavaScript vilket innebär att sidan inte laddas om vid användarens interaktioner.

Design

Webbsidan har en enhetlig grafisk profil inspirerad av spelens visuella stil. Layouten är responsiv och anpassar sig till olika skärmstorlekar med hjälp av media queries och flexibla mått. Det finns även viss anpassning av läsbarhet för användare som använder Light-mode.

Arbetsprocess och implementation

I projektet används en automatiserad arbetsprocess med Node.js och Vite. Vid build-processen så optimeras filer och bilder genom minifiering av kod och komprimering av bilder. Bildoptimeringen resulterade i en minskning av filstorleken men runt 93%.

API: er anropas och hanteras med asynkrona funktioner vilket gör hämtningen och visningen av data mer effektiv.

Animering

Webbsidan innehåller ett antal animeringar som förbättrar användarupplevelsen genom att ge hintar och feedback på vad interaktionen gör, exempelvis förstoring av cards vid hover och visuell feedback vid klick av sökknapp.

6 Analys

Projektet resulterade i en fungerande webbsida som uppfyller projektkraven. Arbetet med detta har gett en ökad förståelse för hur flera externa källor kan integreras, men också vilka utmaningar som kan uppstå.

En av de första utmaningarna var att hitta lämpliga API: er som passade in i den idé som fanns kring vad webbsidan skulle erbjuda för typ av tjänst. Tanken var att presentera mer lore och karaktärsinformation från spelen, och även om API för detta kunde hittas så fick jag inte detta att fungera på grund av begränsningar i API. Därför fick jag tänka om gällande innehåll och kompromissa bort vissa saker vilket resulterade i att Wikipedias API användes som komplement för information vilket inte gav det resultat jag hoppats för webbsidan.

En annan utmaning var att filtrera data fån RAWG API. Deras databas innehåller både många spel och titlar även inom Far Cry som inte var relevanta att visa men som ändå kom upp vid sökning. Arbetet med att filtrera bort oönskade resultat var tidskrävande då jag behövde öka min kunskap både vad i API anropet som genererar vissa resultat och hur jag bäst kunde lösa filtreringen i sökfunktionen. Resultatet blev fungerande, men inte optimalt då jag hade velat begränsa sökresultatet ytterligare beroende vad användaren söker på.

Navigeringen var även detta en utmaning då jag ville att användaren skulle kunna trycka sig tillbaka utan att behöva ladda om sidan eller göra en ny sökning. Detta löstes genom att dölja och visa vissa element beroende på vart användaren kom ifrån. Detta var klurigast att lista ut vart i koden dessa dölja funktioner skulle placeras och var mest tidskrävande i form av testning av kod. Med en överblick på det fungerande resultatet av att visa och dölja element blev det tydligt att denna funktion inte är så svår att lösa när man förstår hur de olika funktionerna fungerar tillsammans.

Valet av en automatiserad arbetsprocess med Node.js och vite har fungerat väl och bidrog till att göra utvecklingen av webbsidan smidigare. Att använda *async/await* gjorde för egen del koden mer lättläst och strukturerad. Dock är laddningstiden av webbsidans innehåll genererat från API: er fortfarande för lång och jag behöver hitta sätt som gör hämtningen mer effektiv.

Att strukturera upp projektet med Sass har fungerat bra främst med uppdelning av partials och variabler som har underlättat för att både felsöka och justera i vissa komponenter.

Förbättringar som kan göras inför liknande projekt är att i ett tidigt skede testa API: er för att säkerställa att design och funktion kan fungera enligt plan, samt att effektivisera hämtning och visning av data ytterligare för att öka prestandan och på så vis även användarvänligheten.

7 Källförteckning

[1] Node.js.org, “Introduction to Node.js”, https://nodejs.org/en/learn/getting-started/introduction-to-nodejs Hämtad 2026-03-17

[2] Kursmaterial: ”Node.js och NPM”, Mittuniveristet. Tillgänglig via Canvas, https://canvas.miun.se/courses/8221/pages/nodejs-och-npm?module_item_id=195258 Hämtad 2026-03-17

[3] Kursmaterial: ”Arbetsprocessen”, Mittuniversitetet. Tillgänglig via Canvas, https://canvas.miun.se/courses/8221/pages/arbetsprocessen?module_item_id=195259 Hämtad 2026-03-17

[4] Kursmaterial: ”Automatisering med Vite”, Mittuniversitetet. Tillgänglig via Canvas. https://canvas.miun.se/courses/8221/pages/automatisering-med-vite?module_item_id=195261 Hämtad 2026-03-17

[5] Ilya Kantor, “Array methods”, https://javascript.info/array-methods Publicerad 2023-12-31. Hämtad 2026-03-17

[6] Kursmaterial: ”Promises”, Mittuniversitetet. Tillgänglig via Canvas, https://canvas.miun.se/courses/8221/pages/promises?module_item_id=198898 Hämtad 2026-03-17

[7] Ilya Kantor, “Async/await”, https://javascript.info/async-await Publicerad: 2025-03-24. Hämtad 2026-03-17

[8] Ilya Kantor, “Error handling, try…catch”, https://javascript.info/try-catch Publicerad: 2025-04-08. Hämtad 2026-03-17

[9] Sass , “Sass Basics”, https://sass-lang.com/guide/ Hämtad 2026-03-17

[10] Sass, “@if and @else”, https://sass-lang.com/documentation/at-rules/control/if/ Hämtad 2026-03-17

[11] Kitty Giraudel, ”Sass Guidelines. An opinionated styleguide for writing sane, maintainable and scalable Sass.” https://sass-guidelin.es/#architecture Hämtad 2026-03-17

[12] W3Schools, “CSS3 Animations”, https://www.w3schools.com/css/css3_animations.asp Hämtad 2026-03-18

[13] Kursmaterial: ”Transition”, Mittuniversitetet. Tillgänglig via Canvas https://canvas.miun.se/courses/8221/files/590722?module_item_id=201201&fd_cookie_set=1 Hämtad 2026-03-18

[14] Kursmaterial: ”Transform”, Mittuniversitetet. Tillgänglig via Canvas, https://canvas.miun.se/courses/8221/files/590727?module_item_id=201202&fd_cookie_set=1 Hämtad 2026-03-18

[15] W3Schools, “Web APIs - Introduction”, https://www.w3schools.com/js/js_api_intro.asp Hämtad 2026-03-18

[16] W3resource, “API Key Guide”, https://www.w3resource.com/computer-programming/api-key-guide.php Publicerad: 2025-01-09. Hämtad 2026-03-18

[17] W3Schools, “What is JSON?”, https://www.w3schools.com/whatis/whatis_json.asp Hämtad 2026-03-18

[18] JSDoc contributors, ”Getting Started with JSDoc 3” https://jsdoc.app/about-getting-started Hämtad 2026-03-18

[19] Wikipedia.org, ”Mashup (web application hybrid)” https://en.wikipedia.org/wiki/Mashup_(web_application_hybrid) Publicerad: 2026-01-26. Hämtad 2026-03-18

[20] W3Schools, “AJAX Introduction”, https://www.w3schools.com/xml/ajax_intro.asp Hämtad 2026-03-18

API-Dokumentation

[21] RAWG, “RAWG Video Games Database API”, https://api.rawg.io/docs/#operation/games_read Hämtad 2026-03-16

[22] MediaWiki.org, ”API:REST API”, https://www.mediawiki.org/wiki/API:REST_API Hämtad 2026-03-16

Bilaga A: Grafisk profil

FRONTIER ATLAS

Bakgrundsfärg: #0C0B0B, RGB: 12, 11, 11

Textbakgrund Dark mode: RGBA: 12, 11, 11, 8

Textfärg: #E3E4DC, RGB: 227, 228, 220

Textbakgrund Light mode: RGBA: 227, 228, 220, 8

Knappar: #455935, RGB: 69, 89, 53

Accent: #853a21, RGB 133, 58, 33

Rubriker: Barlow Condensed, Bold Italic

h1: 48 px, 32 px

h2: 32px

h3: 24 px

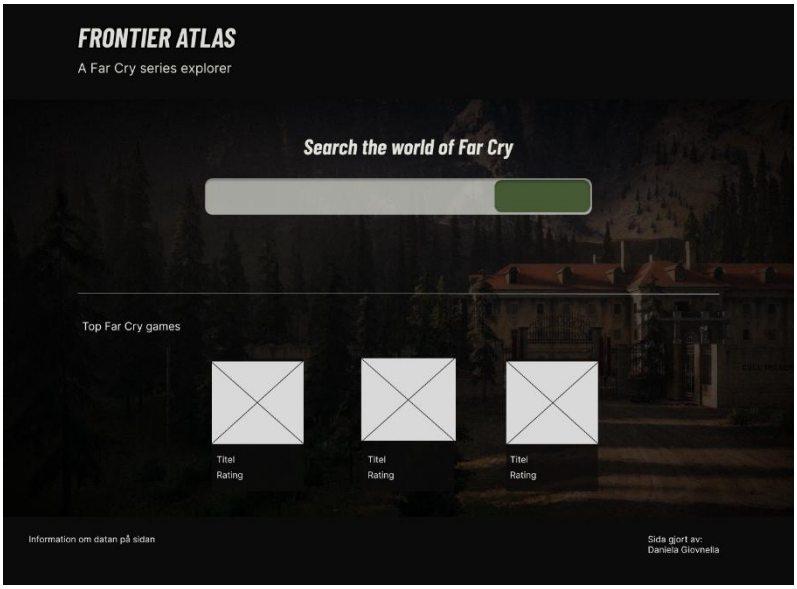
Brödtext: Inter, 16 px

Bilaga B: Grafiska skisser

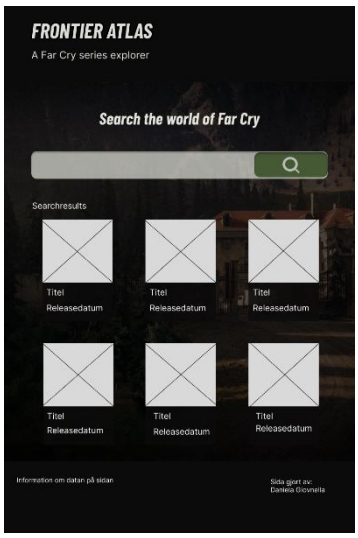
Länk till skisser i Figma:

[Grafiska skisser. Far Cry Wiki]

Utdrag av skisser som fungerat som underlag för utvecklingen av projektet.



Startsida - Desktop



Sökresultat - Tablet

Spelinfo - Mobile